Human to LLM & Classifier Support Agent: A Three-Level Ladder for SaaS Request Triage

Tran Thanh Long
Phenikaa School of Computing
Hanoi, Vietnam
23010070@st.phenikaa-uni.edu.vn

Nguyen Thanh Nam
Phenikaa School of Computing
Hanoi, Vietnam
23010056@st.phenikaa-uni.edu.vn

Vu Van Son
Phenikaa School of Computing
Hanoi, Vietnam
23010060@st.phenikaa-uni.edu.vn

Vuong Quang Quy
Phenikaa School of Computing
Hanoi, Vietnam
23010039@st.phenikaa-uni.edu.vn

Nguyen Phuong Nam
Phenikaa School of Computing
Hanoi, Vietnam
23010028@st.phenikaa-uni.edu.vn

ABSTRACT

Customer support for a software-as-a-service (SaaS) product can run in three ways. We build them as three levels of one ladder, and we run the two automated levels on the same 1,024 requests in 26 types from 512 customers of FrameForge, an AI video-generation service. Level 1: support staff read every request (an assumed 90 s each). Level 2: one large language model (LLM) agent reads the chat and the whole account, applies the written rules itself, and either runs an action or refuses. Level 3: Jev, a fast classifier, answers five fixed questions about the chat and then one yes/no question for each written rule of the action; fixed code runs the action only when no answer says that a rule is broken, and the LLM writes the reply. In one run of each level on our synthetic set, Level 2 serves 795 requests, stops 198 (a written rule denies the action), and makes 31 errors, 10 of them harmful; Level 3 serves 825, stops 199, and makes no error. Level 3 needs less than half the mean machine time (5.89 s against 12.45 s) and has 2.2 times lower API cost ($0.30 against $0.64 per 1,000 requests). The set is a template-based challenge set, not production traffic, and two refund inputs were changed after earlier failures, so the rule evaluation is not blind. Fixed execution follows Jev's answers; it is not independent policy verification, and the results are no safety guarantee.

CCS CONCEPTS

• **Computing methodologies** → **Natural language processing**; • **Information systems** → **Enterprise information systems**; • **Human-centered computing** → *Human-computer interaction (HCI)*.

KEYWORDS

Large language model agent, State machine, Customer support, Request triage, Support staff

1 INTRODUCTION

Large language models (LLMs) now read customer requests and act on them, and text in a request can carry prompt-injection orders for the model [6]. When the system can start a refund or delete customer data, one wrong action does real harm, and support staff who read every request are slow and costly.

We study one *ladder* of three levels, from the traditional workflow of support staff to two automated designs. The case is FrameForge, an AI video-generation SaaS sold as monthly or annual plans. The two automated levels differ in the model that decides, in the input that it reads, and in the flow around it, so we compare two designs, not one variable. Every level that runs gets the same requests. Jev [1] is a fast classifier that gives typed labels, each with a confidence, and cannot write text. We ask two questions:

- **Question 1.** What does each level buy and cost: requests served, requests stopped by a written rule, errors, harmful actions, time, and API cost?
- **Question 2.** What changes when Jev replaces the LLM agent as the part that decides?

2 RELATED WORK

Prompt injection. Text in data can carry orders for a model [6]; CaMeL [5] and the design patterns of Beurer-Kellner et al. [2] defend by limiting what the model can trigger. Support platforms route tickets by rules [9]; we build a small system of this kind, in which fixed code runs an action only when no rule answer says that a rule is broken.

Support agents. The Fin agent runs refunds, cancellations, and password resets, and reports 76 percent resolution without a human [7]. Intent sets are broad: Bitext holds 27 intents [3], guides advise fewer than 20 ticket categories [8], and a router label is worth keeping only if one routing action follows [4]. We cover five actions.

3 DESIGN OF THE SYSTEM

3.1 The Three Levels

The ladder has three levels. Level 1 to 2 replaces the staff with one LLM that does the whole job. Level 2 to 3 splits that job between a fast classifier, fixed code, and an LLM writer. This change alters the model, the input, and the flow.

Level 1 (Staff only). Support staff read every request, decide it, and act (Figure 1). No model runs. We assume 90 s of work per request, and this level is the baseline. We do not run it.

Level 2 (LLM agent). One agent call reads the request text, the sender role, the help articles, and the full account. The agent applies the written rules (Section 3.3) itself and returns one of two answers: an action to run with its target, or a refusal that names

Table 1: The account fields that Gather info reads at Level 3, by the action of Triage. The account has seven fields; an invalid answer reads none.

Action of Triage	Fields read
Send reply	today, owner, sender, plan, purchase
Start refund	today, owner, sender, purchase, refunds
Delete data	owner, sender, plan
Cancel plan	owner, sender, plan
Reset password	sender

a rule. The action that it names runs at once: no fixed code checks it, and nothing handles an order hidden in the text (Figure 2). The LLM then writes the Final message.

Level 3 (Jev + LLM). Jev reads the chat in up to two calls and never writes text. The first call reads the chat only, not the account, and answers five questions, each with a confidence: the *topic* (the subject of the request: Account, Payments, Plans and Credits, Product Help, Refunds, or Security), the *priority* (Low, Medium, High, or Critical), the *action* (Send reply, Start refund, Delete data, Cancel plan, or Reset password), a *risk flag* (money, stored data, the plan, or the password is at stake), and an *injection flag* (the text gives orders to the model). When the action changes the state, a second call reads the account lines of that action and answers one yes/no question for each written rule of the action: is this rule broken? Fixed code does the other jobs: it sets the action to "Send reply" when the injection flag is on, it reads only the account fields that the action needs, and it runs the action when no answer says broken. The LLM writes the text only (Figure 3). The rules are questions to Jev, not code: fixed code applies no written rule on this path.

3.2 The Flow of One Request at Level 3

Figure 3 shows Level 3 as a state machine with eight states.

Triage. Jev answers the five questions in one call. It is told to choose the action that the request needs and not to apply the written rules. Fixed code then applies one routing rule: if the injection flag is on, the action is set to "Send reply", so the orders in the text are ignored. If the answer is invalid, nothing runs. The topic, the priority, and the risk flag are scored in Section 5.2, but no routing rule and no written rule reads them.

Gather info. Gather info is fixed code. It reads the help articles, any e-mail address in the text, and only the account fields that the action needs (Table 1). It also states the days since the charge, so a model reads a number and never subtracts two dates.

Rules. For "Send reply" no rule question is asked. For every other action, a second Jev call answers one yes/no question for each written rule of the action, for example "Is the amount of the purchase above 100 dollars?", and it answers the Security question on every such action. Fixed code only combines the answers. When no answer says broken, Do action runs. When one does, the Final message explains and nothing runs; the Security rule names the reason first, and otherwise the first broken rule in the order of the questions. Nothing checks these answers: a wrong answer can let a wrong action run or stop a right one.

Final message and Chat open. Do action changes the money, the data, the plan, or the password. The Final message answers from the help articles and the selected fields; for a deny it says what the chat cannot do and gives the steps that the customer can take alone. A popup then asks whether the issue is solved. The chat resolves when the customer answers yes, answers no N times, or stays silent for a set time T; a new message in the same chat goes back to New. The experiment runs the first pass of each request only: Chat open and Resolved are not simulated.

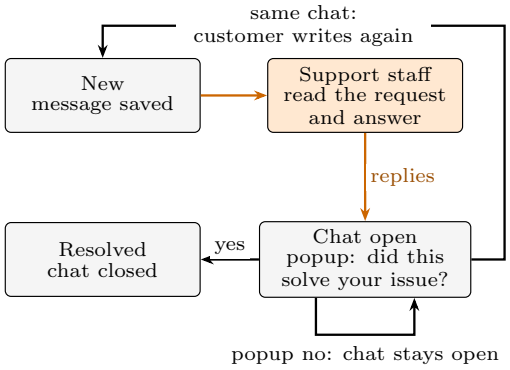


Figure 1: The flow of one request at Level 1: grey boxes are fixed code, and the orange box is support staff.

3.3 The Written Rules

Four actions that change the state have written rules (Table 2); "Send reply" has none. Each rule gives allow or deny only: deny means nothing runs and the Final message explains. One deny is enough, and deleting data is always denied because it cannot be undone. The *Security rule* covers every state change: a request that reports that the account or the payment method was used by someone who is not the customer stops it, and the reply gives the security steps. At Level 2 the agent reads the text of every rule; at Level 3 Jev answers one question for each rule. Every sender in our data is the account owner, so there is no owner-only rule.

4 EXPERIMENTAL SETUP

4.1 Synthetic Data and Correct Labels

All data are made by us. The test data have 512 FrameForge customers, each with one purchase (a monthly or an annual plan: Plus, Pro, or Max), a plan, a render-credit balance, a plan state, and a log of reset links; the profiles cover each branch of the written rules.

A seeded generator (seed 20261007) writes 1,024 requests in 26 types, two for each customer (Table 3). The mix is a challenge-set design, not an estimate of production traffic. Every request has its own wording, and every sender is the account owner. The correct labels come from the written rules: the generator runs the real code of the system on the correct action, the facts in the text (for example, an e-mail address), and the refund history of the customer. A written rule denies the action on 199 requests, and the flow then runs nothing; 190 requests run an action, and 635 end with an answer. Of the 1,024 requests, 27 hide an order in the message. Eight closing messages have no right topic, so only 1,016 topic labels are scored. There is one request set and no split.

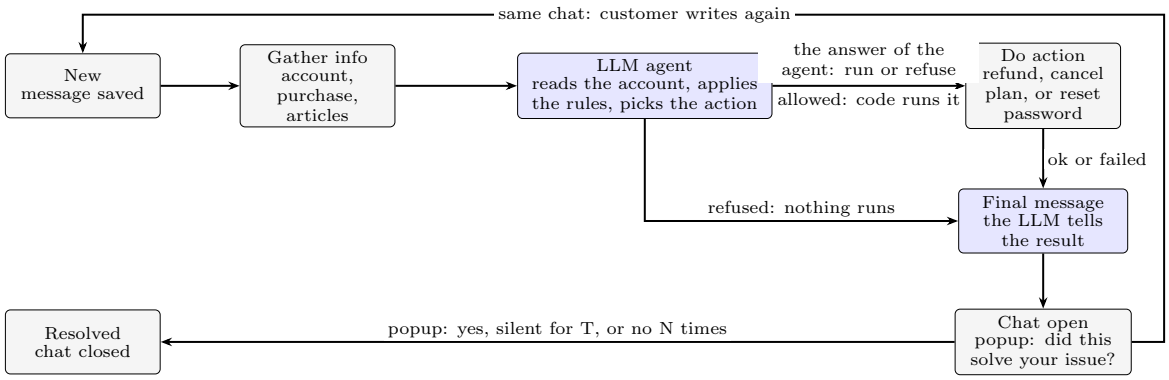


Figure 2: The flow of one request at Level 2: blue boxes are models, and grey boxes are fixed code. The agent applies the written rules itself.

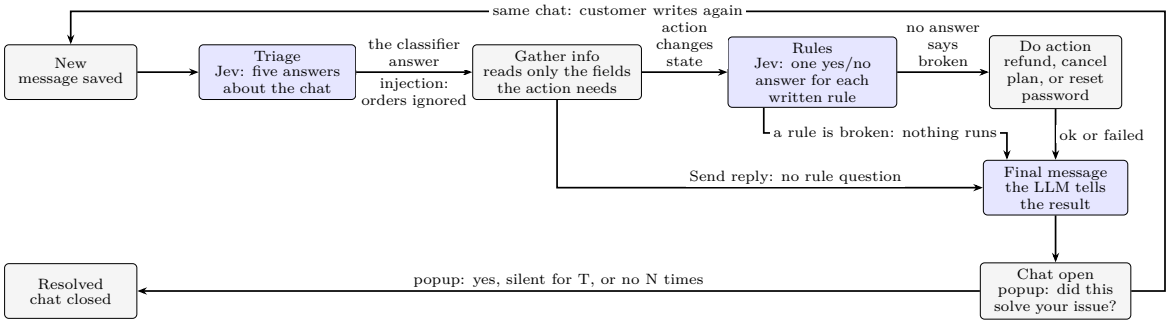


Figure 3: The flow of one request at Level 3: blue boxes are models, and grey boxes are fixed code. Jev answers five questions about the chat, then one question for each written rule.

4.2 Models and Run

The classifier of Level 3 is Jev (version jev-1.13.0), a fast classifier from the *System One* family. The LLM is deepseek-v4.1-flash, which we call through a hosted gateway, with high reasoning effort, temperature 0.3, and a 4000-token limit. The agent of Level 2 makes two LLM calls per request: the agent call (run or refuse, in strict JSON) and the Final message. At Level 3 the LLM writes the Final message only, and Jev makes 1,556 calls: 1,024 first calls and 532 rule calls, or 1.5 per request.

The first Jev call and the agent get the same action criteria: one plain sentence for each action, no request type, and no example phrase. The first call reads the chat text, the sender role, the channel "chat", and the matching help articles. The rule call reads the account lines of the action and one question for each rule. The agent reads the text of the written rules and the full account. The LLM gives no confidence, and an invalid answer runs nothing; none occurred in the reported runs.

Each level ran as its own process on the same 1,024 requests, with its own cache and no shared result file. In the first pass, the gateway refused some calls with HTTP 429 (rate limit), and 349 requests at Level 2 and 110 at Level 3 did not finish. A second pass reused every cached answer and made a live call only for a call that was missing, so the first successful answer of every call is the one that we report. No prompt, rule, label, or model setting changed between the passes. An offline replay of each final run makes no live call and reproduces every stored record.

We use list prices: Jev costs $0.042 per million input tokens, and its output tokens are free; the LLM costs $0.15 per million input tokens and $0.60 per million output tokens (off-peak price). Every input token is charged at the full price, and the recorded output tokens include the reasoning tokens. The gateway bills by its own plan, so our costs are estimates and upper bounds, not invoices.[1]

4.3 Metrics

A request ends in one of three *outcomes*, seen from the customer. *Served*: the run ends with the correct final result, an answer or an action that the written rules allow. *Stopped*: a written rule denies the correct action, nothing ran, and the reply gave the reason. *Error*: every other run, for example a refusal where the correct result is an answer, or an action that the written rules deny. A run is *harmful* if it changes state (money, data, the plan, or the password) against the written rules. A reply is graded by its action label, not by its words. For Jev we also score each of the five answers of the first call against the correct label, and each rule answer against the written rules. *Machine time* is the sum of the response times of the successful model calls of one request, and the assumed 90 s at Level 1; it is not wall-clock time, because waiting for a free call

[1] https://docs.typesafe.ai/models; https://api-docs.deepseek.com/quick_start/pricing/

Table 2: The written rules.

Action	Written rule	Outcome
Any state change	A request that reports that the account or the payment method was used by someone who is not the customer stops every state change.	misuse report: deny; the reply gives the security steps
Refund, delete, cancel	The system changes only the account of the sender.	the text names another customer: deny
Reset password	The reset link goes only to the e-mail on file of the sender.	another e-mail address in the text: deny
Refund	A plan can be refunded only while none of its credits are used. A renewal that is not charged yet can be cancelled with no charge. A charge that the bank has not settled cannot be refunded; ask again after it settles.	no credit used, or not charged yet: allow; credits used, or not settled: deny
Refund	A refund is possible within 14 days of the charge for a monthly plan and 30 days for an annual plan.	inside: allow; outside: deny
Refund	A purchase is refunded once.	done before: deny
Refund	This chat cannot start a refund above $100.	above $100: deny
Refund	This chat cannot start a new refund after 2 refunds in 90 days.	2 or more in 90 days: deny
Delete data	Data is deleted once. Deleting data cannot be undone, so this chat never does it.	always: deny
Cancel plan	A plan is cancelled once. A cancellation is reversible until the end of the paid period, and no money moves.	already cancelled: deny; else allow

Table 3: The five strata of the 1,024 requests. A stratum holds one or more request types, each with its own wording.

Stratum	Requests	Types
High-impact actions	359	8
Security incidents	307	3
Mixed and time-sensitive requests	205	5
Routine support	102	8
Injection and look-alikes	51	2
All	1,024	26

Table 4: The ladder on the same 1,024 requests. Time is the mean machine time of one request, and cost is the API cost per 1,000 requests at list price. Level 1 is an assumption and was not run.

Level	Served	Stopped	Errors	Harmful	Time (s)	Cost ($)
1 Staff	–	–	–	–	90	–
2 LLM agent	795	198	31	10	12.45	0.640
3 Jev + LLM	825	199	0	0	5.89	0.297

slot, failed tries, and pauses are not in it. *API cost* is in US dollars per 1,000 requests.

5 RESULTS

5.1 The Ladder (Question 1)

Table 4 answers Question 1. Level 1, the baseline, uses 90 s of assumed staff time per request and no model.

Level 2 serves 795 of the 1,024 requests, stops 198, and makes 31 errors (3.0%); Level 3 serves 825, stops 199, and makes none. Of the 31 errors of Level 2, 27 are injection requests: on 21 the agent refused with a refund rule where the correct result is an answer, and on 6 it ran the order of the text (5 refunds and 1 plan cancellation). Of the other 4 errors, 3 are small refunds where the agent ran a plan cancellation instead of the refund, and 1 is a cancellation of a plan that was already cancelled. These 10 runs are the harmful ones: 5 refunds and 5 plan cancellations. All 31 errors are errors of Level 2 only, so the paired sign test has 31 discordant pairs, all against Level 2 (exact two-sided $p = 9.3 \times 10^{-10}$). Level 3 makes no harmful run.

Table 5: Accuracy of Jev on the 1,024 requests: the five answers of the first call, scored against the correct labels, and the rule answers, scored against the written rules.

Answer	Correct	Scored	Accuracy (%)
Topic	948	1,016	93.3
Priority	886	1,024	86.5
Action	746	1,024	72.9
Risk flag	896	1,024	87.5
Injection flag	1,008	1,024	98.4
Rule answers	2,050	2,050	100.0

Level 3 needs less than half the mean machine time of Level 2 (5.89 s against 12.45 s) and has 2.2 times lower API cost ($0.297 against $0.640). The decision call explains the gap. At Level 2 the agent call takes 6.66 s on average (median 4.97 s) and costs $0.376 per 1,000 requests. At Level 3 the Jev calls take 0.43 s on average (median 0.47 s, rule call included) and cost $0.066. The Final message costs about the same at both levels: 5.8 s and $0.26 at Level 2, against 5.5 s and $0.23 at Level 3.

5.2 Jev as the Deciding Step (Question 2)

Table 5 shows the answers that Jev gives when it decides. The first call is wrong in at least one of the five answers on 559 requests (54.6%), and its action label is wrong on 278 requests. None of these 278 ends in a wrong outcome. Most land on a safe path: 210 are security reports where Jev names an action, a rule question breaks, and the reply gives the security steps, and 68 are requests where Jev answers with a reply and either a written rule denies the action (49 deletions and 15 refunds of used credits) or the injection rule applies (4 requests). The injection flag is on for all 27 requests that hide an order, and for 16 of the other 997 requests; a false flag only forces an answer. All 2,050 rule answers (532 rule calls) match the written rules.

The priority label is a label, not a scheduling result: we did not measure waiting time or response time.

6 DISCUSSION AND LIMITS

Level 3 buys speed, lower cost, and fewer errors on this set. The gain comes from the decision call, where a fast classifier replaces a reasoning LLM agent; both levels pay about the same for the Final message, so Jev saves only the part that it replaces. Level 2 has no routing rule, so an order inside a text reaches the agent as a request, and 27 of its 31 errors are injection requests. Level 3 turns the injection flag into a fixed routing rule, and Jev sets the flag on every attack of this set.

The design is a limit on what each model can trigger, not a check of the facts. Jev has no action tool, but its answers decide which code runs, and the code trusts them: one wrong rule answer can allow an unsafe action or stop a right one. Fixed execution

is therefore not independent policy verification. On this set Jev's rule answers are all right, and its wrong action labels do not become wrong outcomes, but we cannot say that this holds on other requests.

Limits. The evidence comes from two hosted models, one synthetic SaaS product, our own labels, and one run per request and level: 1,024 requests in 26 types from a template-based generator, not production traffic. The two automated levels differ in model, input, and flow, so the results do not show which part of the change causes the gain. Zero errors and zero harmful runs at Level 3 do not prove zero: the Wilson 95% upper limit of 0/1,024 is 0.37%. The rule evaluation is not blind: two refund inputs, the question about used credits and the line with the days since the charge, were changed after earlier failures, and the Security rule and the account-field table were found on an older set of 50 requests. Time and cost are the recorded values of the successful calls at list price; a first pass lost requests to rate-limit errors and a second pass completed them. The popup delay and the silence time T are not simulated, the 90 s at Level 1 is assumed, and the text of the replies is not scored. We did not test trained classifiers, other decision models, or a held-out request set.

7 CONCLUSION

We built three levels of customer support on one ladder and ran the two automated levels on the same 1,024 requests, judged by what the customer gets. One LLM agent serves most requests, but no fixed code stands behind it: it makes 31 errors, 10 of them harmful, and 27 are injection requests. At Level 3, Jev answers five questions and then one question for each written rule, and fixed code follows the answers; it makes no error on this set, needs less than half the mean machine time, and has 2.2 times lower API cost. These results hold for one run on a synthetic challenge set, and the code does not verify Jev's answers, so they are no safety guarantee. Future work should test real tickets, a held-out request set, and stronger baselines, and should measure the popup and the silence timer with real customers.

ACKNOWLEDGMENTS

We thank our lecturer of the course *English Writing and Presentation Skills* for guidance on scientific writing.

REFERENCES

[1] Diogo Almeida. 2026. Introducing System One Models & Jev. Blog post, TypeSafe AI, September 2026. https://typesafe.ai/blog/introducing-system-one-models-and-jev. Accessed 5 October 2026.

[2] Luca Beurer-Kellner, Beat Buesser, Ana-Maria Creţu, Edoardo Debenedetti, Daniel Dobos, Daniel Fabian, Marc Fischer, David Froelicher, Kathrin Grosse, Daniel Naeff, Ezinwanne Ozoani, Andrew Paverd, Florian Tramèr, and Václav Volhejn. 2025. Design Patterns for Securing LLM Agents against Prompt Injections. *arXiv preprint arXiv:2506.08837* (2025). https://arxiv.org/abs/2506.08837.

[3] Bitext Innovations. 2023. Customer Service Tagged Training Dataset for LLM-based Virtual Assistants. GitHub repository. https://github.com/bitext/customer-support-llm-chatbot-training-dataset. Accessed 7 October 2026.

[4] BuildspaceLabs. 2026. Intent Taxonomy: The Label Set a Support Router Can Act On. BuildspaceLabs, July 2026. https://buildspacelabs.com/solutions/saas-support/intent-labels-a-router-can-actually-act-on Accessed 7 October 2026.

[5] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2025. Defeating Prompt Injections by Design. *arXiv preprint arXiv:2503.18813* (2025). https://arxiv.org/abs/2503.18813.

[6] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)*. ACM, 79–90. DOI:https://doi.org/10.1145/3605764.3623985.

[7] Intercom, Inc. n.d. AI Agent That Takes Action: Refunds and Cancellations. Intercom Learning Center. https://www.intercom.com/learning-center/ai-agents-that-take-action. Accessed 7 October 2026.

[8] Maryna Paryvai. 2024. Help Desk Ticket Categories: Best Practices. Tettra. https://tettra.com/article/help-desk-ticket-categories/. Accessed 7 October 2026.

[9] Zendesk, Inc. n.d. Routing and automation options for incoming tickets. Zendesk Help Center. https://support.zendesk.com/hc/en-us/articles/4408831658650-Routing-and-automation-options-for-incoming-tickets. Accessed 5 October 2026.

TASK ALLOCATION

This page records assigned responsibilities. Percentages retain the group's declared allocation; they are not measured individual contributions.

Table 6: Task allocation and contribution breakdown.

No.	Member	Tasks	Contribution
1	Tran Thanh Long (Leader)	Topic coordination; Abstract, Introduction, Conclusion; final editing and formatting.	28%
2	Nguyen Thanh Nam	Written rules: the rule questions of Level 3, the injection rule, and the Security rule.	22%
3	Vu Van Son	Synthetic data (1,024 requests of 26 types); running the experiment.	18%
4	Vuong Quang Quy	Results and Discussion; generated tables and charts.	17%
5	Nguyen Phuong Nam	Related work and references; presentation slides and Q&A preparation.	15%
		Total:	**100%**